

# CAutomation Project Engineering Contract

Project-level engineering contract for the CAutomation platform

**Status:** Project-level contract template for CAutomation. This document defines platform-wide business scope and rules. Module-specific details belong in module-level contracts.

## 1. Document Purpose

This contract defines how CAutomation applications shall be engineered at platform level. It applies to all modules and all downstream AI-generated artifacts unless a later approved platform contract explicitly changes it.

- Translate the Project Client Contract into platform-wide engineering rules.
- Define shared architecture, technology stack, security, API, testing, logging, deployment, and AI generation standards.
- Exclude module-specific requirements, database tables, endpoints, screens, and workflows unless they are shared platform conventions.

## 2. Engineering Principles

- Contract-first: approved human-authored contracts are the source of truth.
- Validation-first: invalid or insufficient inputs must fail early before downstream generation.
- Traceability: generated artifacts and reports must reference their source contracts and executions.
- Deterministic execution: pipeline tasks must produce predictable, inspectable outputs.
- Modular architecture: project-level rules are shared; module-level logic stays isolated.
- Small, testable task units: implementation, unit tests, functional tests, and validation stay aligned.

## 3. Platform Architecture

CAutomation is modeled as a multi-module project executed by an AI Engine. The project contains project-level contracts and module-level contracts. Pipelines consume normalized inputs and produce reports and deliverables.

- CAutomation project: platform-level configuration, contracts, modules, normalized input, reports, and deliverables.
- Modules: independently specified functional areas with their own SRS and ATS.
- AI Engine: pipeline/task execution framework for normalization, context building, generation, validation, and reporting.
- Normalized input: canonical downstream representation that preserves project-level versus module-level source distinctions.

## 4. Technology Stack

| Area                  | Canonical choice                                                                       |
|-----------------------|----------------------------------------------------------------------------------------|
| Backend language      | Python                                                                                 |
| Backend web framework | FastAPI                                                                                |
| Frontend              | React with TypeScript                                                                  |
| Database              | PostgreSQL                                                                             |
| ORM / database access | SQLAlchemy, with migrations handled by Alembic when database migrations are introduced |
| Authentication        | JWT-based authentication with server-side authorization checks                         |

|                  |                                                                                             |
|------------------|---------------------------------------------------------------------------------------------|
| Testing          | pytest for Python; frontend tests to be defined when frontend implementation begins         |
| Documentation    | Markdown as editable source where practical; PDF as primary supported contract input format |
| Containerization | Docker                                                                                      |
| CI/CD            | GitHub Actions or equivalent repository-based automation                                    |

## 5. Project Structure

The canonical project hierarchy must make the separation between project-level contracts and module-level contracts explicit.

- CAutomation/projects/CAutomation/input/project/contracts/client/ for project client contracts.
- CAutomation/projects/CAutomation/input/project/contracts/engineering/ for project engineering contracts.
- CAutomation/projects/CAutomation/input/modules//contracts/ for module-level SRS and ATS documents.
- CAutomation/projects/CAutomation/normalized\_input/project/contracts/ for normalized project-level contracts.
- CAutomation/projects/CAutomation/normalized\_input/modules//contracts/ for normalized module-level contracts.

## 6. Backend Standards

- Use layered backend design: API routes, DTOs/schemas, services, repositories, domain models, and persistence.
- Business rules belong in services or domain-level components, not directly in route handlers.
- Repository code owns persistence access and must not contain UI or presentation logic.
- Validation must be explicit and testable.
- Configuration must be externalized and readable by pipeline tasks.
- Use structured errors that can be consumed by UI, tests, and reports.

## 7. Frontend Standards

- Use React with TypeScript for user-facing web application screens.
- Use clear component boundaries for pages, forms, tables, status displays, review screens, and report views.
- Frontend must not enforce security alone; backend authorization remains authoritative.
- Forms must present validation errors clearly and preserve user intent.
- UI patterns for clients, projects, modules, pipelines, executions, reports, and deliverables should be reusable across modules.

## 8. Database Standards

- Use PostgreSQL for persistent platform data.
- Use stable primary keys and explicit foreign keys for ownership relationships.
- Preserve tenant/client/project/module boundaries in the data model.
- Use timestamps for auditable records where relevant.
- Represent execution and validation outcomes as structured records.
- Database schema details belong in module-level ATS documents unless they are shared platform tables or conventions.

## 9. API Standards

- Use RESTful HTTP APIs unless a later approved contract defines a different protocol.

- Use consistent JSON request and response bodies.
- Include stable error formats with machine-readable codes and human-readable messages.
- Protect all non-public endpoints with authentication and authorization.
- Support pagination for list endpoints that can grow.
- Use versioning when API compatibility must be preserved across releases.

## 10. Security Principles

- Authenticate users before allowing access to client, project, module, pipeline, execution, report, or deliverable data.
- Authorize every operation server-side using role and ownership rules.
- Prevent cross-client and cross-project data leakage.
- Protect secrets outside source code and generated artifacts.
- Record auditable actions for sensitive operations.
- Do not allow AI-generated code to weaken platform security rules.

## 11. Quality Standards

- Every pipeline task must have implementation, unit tests, functional tests, and validation coverage where applicable.
- Workflow specification, implementation, configuration, tests, and documentation must remain synchronized.
- Tests should be executable from task and pipeline levels.
- Validation results must be structured and suitable for later web GUI consumption.
- Documentation must describe implemented behavior, not speculative future features.

## 12. Logging and Monitoring

- Use structured logging rather than ad hoc print statements.
- Pipeline and task executions must produce reports with status, inputs, outputs, validation results, and errors.
- Failures must clearly identify the failed task and the reason.
- Reports must be designed for both human review and future GUI consumption.
- Execution history must be retained where it supports auditability and troubleshooting.

## 13. AI Generation Standards

- AI may only generate from approved, normalized, and traceable source inputs.
- AI must not invent missing requirements, permissions, database entities, APIs, workflows, or deliverables.
- Generated artifacts must identify their source contracts or context package.
- Validation gates are mandatory before applying or accepting generated artifacts.
- Pipeline outputs should be immutable records of an execution.
- Ambiguity must be reported as a validation issue rather than silently resolved.

## 14. Cross-Module Engineering Rules

- Shared platform concepts must be defined once at project level and reused by modules.
- Module-specific requirements must remain inside module-level SRS documents.
- Module-specific technical design must remain inside module-level ATS documents.
- Modules must follow the same authentication, authorization, API, testing, logging, and reporting conventions.

- Cross-module dependencies must be explicit and justified.

## **15. Deployment Principles**

- Use environment-specific configuration for local, test, staging, and production contexts.
- Do not hard-code secrets, credentials, or environment-specific paths.
- Containerization should support reproducible setup and execution.
- CI/CD should run tests and validation gates before promotion.
- Deployment details may evolve, but must not contradict the canonical technology stack without approval.

## **16. Engineering Glossary**

Authoritative technical terms include AI Engine, Pipeline, Task, Normalized Input, Context Package, Validation Gate, Execution Report, Artifact, Module, Project, Client, DTO, Service, Repository, and Contract. Module-level documents may extend this glossary but must not redefine platform-level terms inconsistently.